

Merge array explanation

1. Explain in merge array in c++: void mergeArray(vector<int> &arr,int low ,int mid,int high) { int leftArraySize = mid-low+1; int rightArraySize = high - mid; vector<int> leftArray(leftArraySize),rightArray(rightArraySize);2
2. what is the difference between leftArray(leftSize) and leftArray[leftSize].5



1. Explain in merge array in c++:

```
void mergeArray(vector<int> &arr,int low ,int mid,int high)
{
    int leftArraySize = mid-low+1;
    int rightArraySize = high - mid;
    vector<int> leftArray(leftArraySize),rightArray(rightArraySize);
```

Let's break down this part of the `mergeArray` function line by line — it's part of the **Merge Sort** algorithm in C++.

Function header

cpp

```
void mergeArray(vector<int> &arr, int low, int mid, int high)
```

- `vector<int> &arr` → the array (vector) we are sorting; passed **by reference** so changes affect the original array.
- `low` → starting index of the left subarray.
- `mid` → ending index of the left subarray (and one before the right subarray).
- `high` → ending index of the right subarray.

The goal is to **merge two sorted subarrays**:

less

```
Left part: arr[low ... mid]
Right part: arr[mid+1 ... high]
```

into a **single sorted array**.

Line 1: Find sizes of the two subarrays

cpp

```
int leftArraySize = mid - low + 1;
```

```
int rightArraySize = high - mid;
```

Example:

```
ini
```

```
low = 0, mid = 2, high = 5
```

Then:

```
ini
```

```
leftArraySize = 2 - 0 + 1 = 3 → arr[0], arr[1], arr[2]
rightArraySize = 5 - 2 = 3 → arr[3], arr[4], arr[5]
```

So each subarray's length is calculated correctly.



Line 2: Create temporary subarrays

```
cpp
```

```
vector<int> leftArray(leftArraySize), rightArray(rightArraySize);
```

This creates **two temporary vectors**:

- **leftArray** with **leftArraySize** elements
- **rightArray** with **rightArraySize** elements

These are used to **store copies** of the left and right halves from the main array **arr**.

Next steps (not shown yet, but typically follow)

Usually, you then copy data into these temporary arrays:

```
cpp
```

```
for (int i = 0; i < leftArraySize; i++)
    leftArray[i] = arr[low + i];

for (int j = 0; j < rightArraySize; j++)
    rightArray[j] = arr[mid + 1 + j];
```

Then merge them back:

cpp

```

int i = 0, j = 0, k = low;
while (i < leftArraySize && j < rightArraySize) {
    if (leftArray[i] <= rightArray[j])
        arr[k++] = leftArray[i++];
    else
        arr[k++] = rightArray[j++];
}

```

Copy leftovers:

cpp

```

while (i < leftArraySize)
    arr[k++] = leftArray[i++];
while (j < rightArraySize)
    arr[k++] = rightArray[j++];

```

Summary

So these first lines:

cpp

```

int leftArraySize = mid - low + 1;
int rightArraySize = high - mid;
vector<int> leftArray(leftArraySize), rightArray(rightArraySize);

```

are **setting up two temporary arrays** to hold the **left and right halves** of your vector so that they can be **merged back in sorted order**.

Would you like me to show the **full merge function** with these lines completed and commented?



2. what is the difference between `leftArray(leftSize)` and `leftArray[leftSize]`

Excellent question — and a **very important** one in C++!

Let's carefully break it down 🙋

🧩 1 `vector<int> leftArray(leftSize);`

✅ This is a **constructor call**.

It **creates a vector** named `leftArray` that has `leftSize` elements.

- Each element is automatically initialized to `0` (for integers).
- The parentheses `()` mean:
"Construct this vector **with** this many elements."

Example:

cpp

```
int n = 5;  
vector<int> leftArray(n);
```

Now:

ini

```
leftArray = [0, 0, 0, 0, 0] // size = 5
```

So `leftArray(i)` is used **when creating** the vector.

🧩 2 `leftArray[leftSize]`

⚠️ This is an **indexing operation**.

It **accesses an element** in an existing vector named `leftArray`.

If `leftArray` was already created, then:

cpp

leftArray[index]

refers to the element **at that index** (0-based).

Example:

cpp

```
vector<int> leftArray(5); // size = 5
leftArray[2] = 10;       // access index 2
```

✅ Works fine — changes the 3rd element.

⚠️ But if you write:

cpp

```
leftArray[5] = 10;
```

You get **undefined behavior**, because index **5** is **out of range** (valid indices are 0–4).

🧠 Summary Table

Syntax	Meaning	When used	Example
<code>vector<int> leftArray(leftSize);</code>	Create a vector with leftSize elements	During declaration	<code>vector<int> a(10);</code>
<code>leftArray[leftSize]</code>	Access element at position leftSize	After declaration	<code>a[3] = 7;</code>

🧩 Analogy

Think of it like this:

cpp

```
vector<int> leftArray(leftSize); // Build a box with leftSize compartments
leftArray[i] = 5;                // Put value 5 in compartment i
```

So:

- `()` → “build vector with N slots”
- `[]` → “access a slot by index”

Would you like me to show an example where using `[]` instead of `()` causes an error when declaring the vector?